Recursividade

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

Tópicos

- Conceitos
- Exemplos
- Exercícios

Conceitos

- Procedimento recursivo: possui em sua descrição uma ou mais chamadas a si mesmo
  - Chamada recursiva: realizada a partir do próprio procedimento
  - Chamada externa: realizada de um local externo ao procedimento
- Procedimento não recursivo: todas as chamadas são externas, ou seja, não há chamada recursiva

Conceitos

- Para todo procedimento recursivo, existe um correspondente não recursivo
  - Nem sempre é fácil eliminar recursões
- Vantagens da recursividade
  - Procedimentos mais concisos: objetivos, curtos
    - Para alguns problemas o uso da recursão é natural: fatorial, Fibonacci, Torre de Hanói
  - Demonstração da corretude/correção mais fácil
    - Indução matemática
- Desvantagem: pode ser menos eficiente
  - Empilhamento/desempilhamento

Exemplos

1. Cálculo de $n! = \begin{cases} 1 & se\ n \leq 1 \\ n(n-1)! & se\ n > 1 \end{cases}$

$$\begin{array}{l} \texttt{função}\ fat(i) \\ \quad fat(i) := \texttt{se}\ i \leq 1\ \texttt{então}\ 1\ \texttt{senão}\ i \times fat(i-1) \end{array}$$

versão equivalente:

```
Fat(i)
    se i ≤ 1 então
            retorne 1;
    senão
            retorne i*Fat(i-1);
```

Exemplos

1. (cont.)

```
Fat(i)
    se i ≤ 1 então
        retorne 1;
    senão
        retorne i*Fat(i-1);
```

Condição de parada

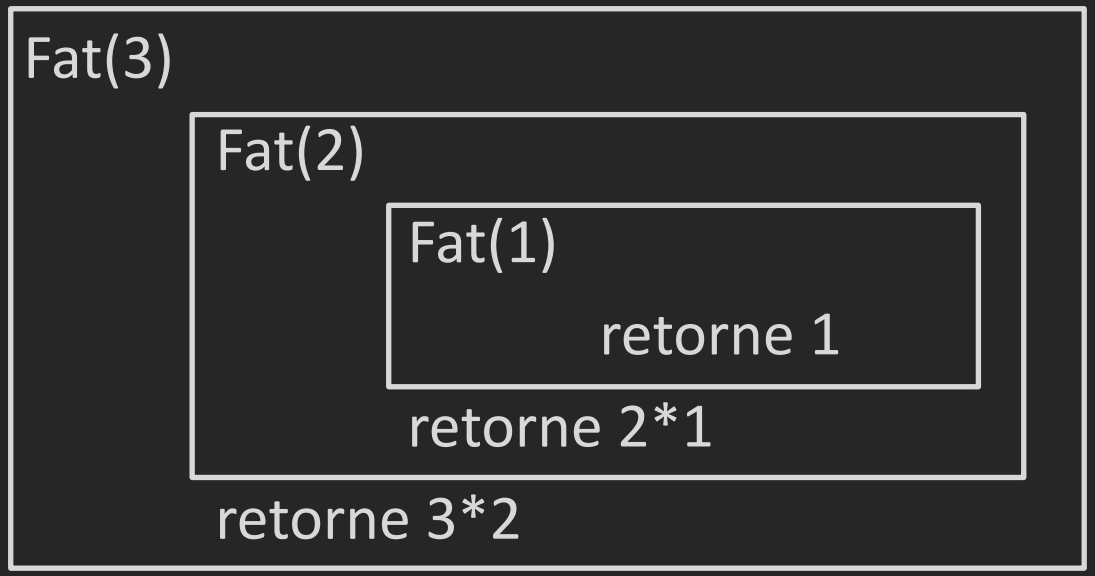
Chamada recursiva

```
FunçãoXYZ(...)
    ...
    Leia n;
    fat_n :=Fat(n);
    soma := fat_n + .... ;
    Imprima soma;
    ....
```

Chamada externa

Exemplos

1. (cont.)
   - Exemplo de execução de Fat(3)



Exemplos

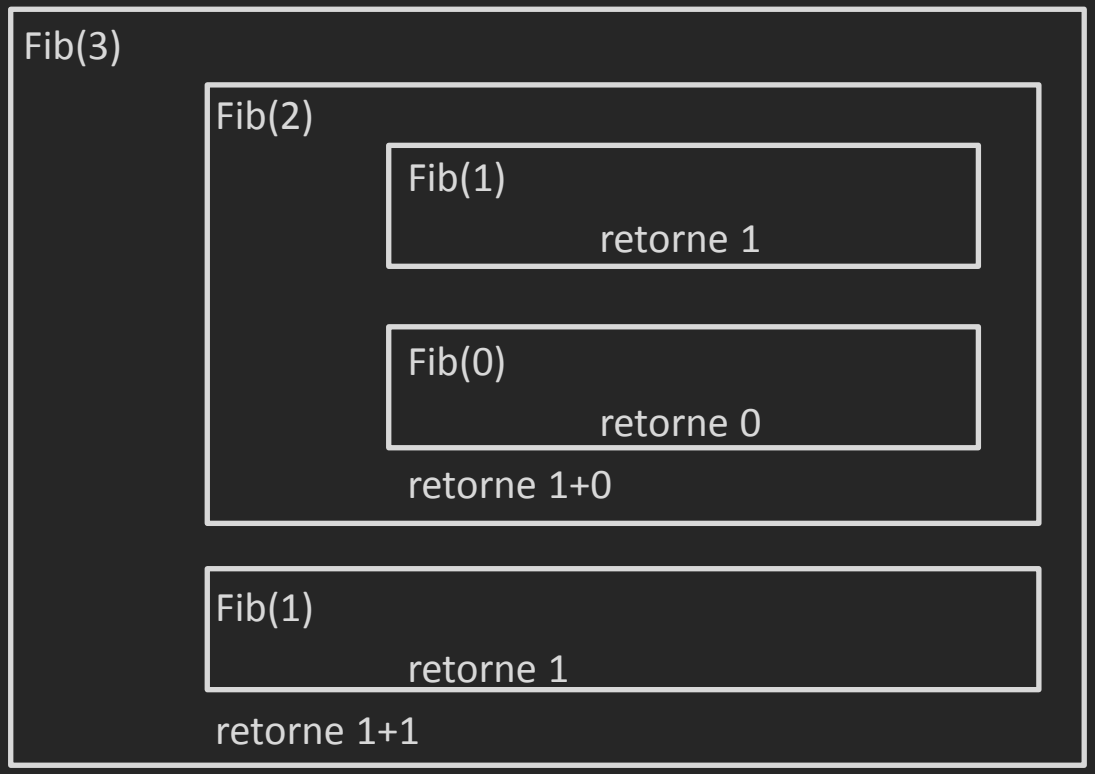
2. Cálculo do n-ésimo número de Fibonacci, onde $n \in \mathbb{N}$.

$$\mathrm{F}(n) = \begin{cases} n & se\ n \leq 1 \\ F(n-1) + F(n-2) & se\ n > 1 \end{cases}$$

```
Fib(n)
    se n ≤ 1 então
            retorne n;
    senão
            retorne Fib(n-1) + Fib(n-2);
```

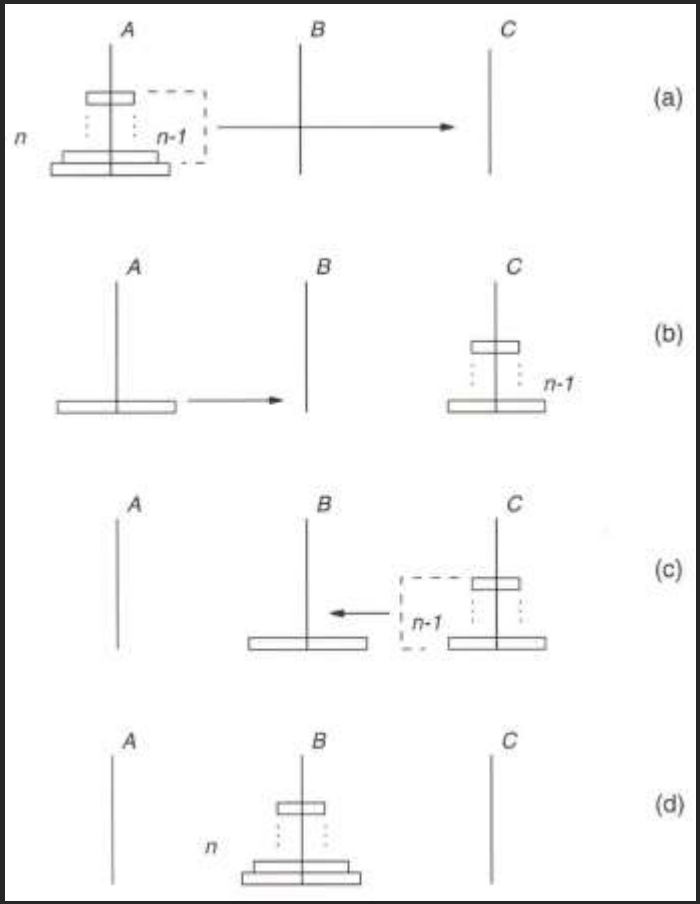
Exemplos

2. (cont.)
   - Exemplo de execução de Fib(3)



Exemplos

3. O Problema da Torre de Hanói com n discos.



Exemplos

3. (cont.)

```
procedimento hanoi(n, A, B, C)
se n > 0 então
    hanoi(n − 1, A, C, B)
    mover o disco do topo de A para B
    hanoi(n − 1, C, B, A)
```

Exercícios

1. Desenvolva algoritmos recursivos para as seguintes situações a seguir:
   a. Inverter uma seqüência com n elementos.
   b. Verificar se uma palavra com n elementos é um palíndromo. Ex.: “somos”, “subinoonibus”.
   c. Somar os números de um conjunto com n elementos.
   d. Verificar se o número x está em um conjunto com n elementos.
   e. Encontrar o maior número de um conjunto com n elementos.
   f. Realizar a busca binária, que é utilizada quando o vetor de entrada está ordenado. A idéia do algoritmo é comparar a chave de busca x com o elemento do meio do vetor para decidir se o elemento x foi encontrado, ou se é necessário continuar a pesquisa nos subvetores à direita ou à esquerda do elemento do meio.

Exercícios

2. Faça versões iterativas dos algoritmos do exercício 1 eliminando as recursões existentes.
3. Implemente em C as versões recursivas e iterativas dos algoritmos desenvolvidos e compare o tempo de execução entre essas duas abordagens. Observação: essa comparação faz sentido quando o tamanho das entradas n é bastante grande.